Papers behind Klaro: short summaries

Study guide 2 of 3. The report has 19 references: 12 papers and 7 tool pages.

Read this first. We did not publish a research paper ourselves. These are the papers *cited* in Chapter 2 of the report (Background Study). For each one: what it did, what it tells us, and exactly how Klaro uses it. The first part of each entry matches what the report says. Extra detail marked † comes from general knowledge of the paper, not from the report, so open the paper if a teacher asks for specifics.

Remember: Chapter 2 story: (A) people already analysed tenders, but only English and mostly with word counting; (B) long documents are hard for transformers; (C) small models make things up; (D) offline translation exists. Nobody had combined offline translation + small summary model + page-level checking on a laptop. That is the gap Klaro fills.

1 A. Earlier work on tender documents

1. Beason et al., 2021 (SMU Data Science Review)

Automated analysis of RFPs using NLP for the technology domain.
Processed a large set of U.S. Department of Defense and Homeland Security RFPs to spot technology trends.
Tells us: RFPs have no common layout and are noisy (contents pages, boilerplate, legal text), so cleaning comes first. Readers prefer exact structured facts over a free-flowing summary.
Klaro uses: skips contents pages; fixed nine headings.

2. Trappey et al., 2022 (J. Global Information Management)

Intelligent RFQ summarization using NLP, text mining and machine learning.
Summarised more than 1,300 power-transformer tenders. Found key terms with **TF-IDF**, grouped sentences by topic, picked top sentences with **TextRank**.
Tells us: keeping the top 35% of sentences kept about 80% of important requirements and cut reading time to about 29%.
Klaro uses: summarise topic by topic (our nine headings, two question sets per chunk) and aim for a real cut in length. (This is also why the context-size bug mattered: our summaries were not getting shorter.)
Difference: theirs is extractive (picks sentences); Klaro uses a language model to write.

3. Knisely and Pavliscsak, 2023 (Scientometrics)

Research proposal content extraction using NLP and semi-supervised clustering.
Sorted research proposals into themes. A few proposals labelled by experts guided the sorting of the rest.
Tells us: a little human labelling beats none at all.
Klaro uses: this is the reason we built the golden reference set (and why we say it still needs human verification).

2 B. Reading very long documents

4. Vaswani et al., 2017 (NeurIPS): "Attention is all you need"

Introduced the **Transformer**, the design behind nearly all modern language models, built on self-attention.
Tells us: attention compares every token with every other token, so cost grows quickly (quadratic†) as text gets longer.
Klaro uses: the reason to cut an RFP into chunks of about 1,800 words instead of reading it whole.

5. Beltagy, Peters, Cohan, 2020: Longformer

Changes attention so cost grows roughly linearly: each token looks at a local window plus a few global tokens†.
Tells us: thousands of words can be read in one pass.
Klaro: we cannot use it. A small local model like Qwen3.5 on a laptop cannot read an 84-page RFP in one go, so we chunk and merge.

6. Douzon et al., 2023 (ICDAR workshops)

Long-range transformer architectures for document understanding.
Used the page layout of business documents, so a table value stays linked to its heading.
Klaro uses: the idea of keeping structure and page position. We keep page numbers on every chunk and fact. (We lose table layout, which is a stated limitation.)

3 C. Small models and made-up answers

7. Ji et al., 2023 (ACM Computing Surveys)

Survey of hallucination in natural language generation.
A survey of why and how text generators state things that are not in the source.† It separates statements that contradict the source from statements that cannot be checked against it.
Klaro uses: the definition of "made-up information" in our nomenclature and our hallucination check.

8. Maynez et al., 2020 (ACL)

On faithfulness and factuality in abstractive summarization.
Found that abstractive summaries often contain content not supported by the document, and that common overlap scores like ROUGE hardly measure this†.
Klaro uses: the reason to check every point against the extracted facts and to be honest that our checks compare words only.

9. Liu et al., 2024 (TACL): "Lost in the middle"

Tells us: models use information at the start and end of a long input better than information in the middle†.

Klaro uses: another reason for chunks, and for our context-size finding: long inputs hurt in practice, not only in theory. (Our G3 run lost facts when given a very long fact list, which fits this idea, but we did not prove that is the cause.)

10. Lewis et al., 2020 (NeurIPS): Retrieval-augmented generation (RAG)

First retrieve the relevant passages, then give only those to the generator.
Klaro uses: not RAG itself. Our remedy is different: force a page citation for every point and verify each point against collected facts. RAG remains an option for later (search over past proposals).

11. Hu et al., 2022 (ICLR): LoRA

Freeze the big model and train small low-rank add-on matrices.
Tells us: far fewer trainable numbers and much less GPU memory than full fine-tuning (for GPT-3 the paper reports about 10,000 times fewer trainable parameters†).
Klaro uses: the fine-tuning kit. It is the only realistic route on an 8 GB card. Output is a small "adapter".

4 D. Machine translation

12. Tiedemann and Thottingal, 2020 (EAMT): OPUS-MT

Open, small translation models, one per language direction, from Helsinki-NLP (built on the Marian toolkit†).
Klaro uses: `opus-mt-de-en` (about 301 MB) to translate German RFPs, and `opus-mt-en-de` for the German summary view. Small enough to run offline on a laptop.
Note: we cite it for what it is. We have not measured its quality ourselves and no German speaker has judged it.

5 Tool references (not papers)

Ollama	Runs open models locally and serves them on `127.0.0.1:11434`. Our model runner.
Qwen3.5 (4B)	The model, tag `qwen3.5:4b`, about 3.4 GB.
Electron	Desktop app framework (Chromium + Node). We follow its security guidance.
FastAPI	Python web framework for the local API.
Pydantic	Checks that model output has the right structure.
pypdf	Reads text from PDFs page by page.
LangGraph	Workflow library we *chose not to use*: our steps are a fixed sequence, a few lines of Python run them.

6 One-line cheat sheet

Beason 2021	RFPs are messy; structured facts beat free summaries.
Trappey 2022	TF-IDF + TextRank on 1,300 tenders; 35% of sentences keeps about 80% of requirements.
Knisely 2023	A few expert labels improve clustering a lot.
Vaswani 2017	Transformer; attention gets expensive with length.
Longformer 2020	Cheaper attention for long text.
Douzon 2023	Use document layout in long documents.
Ji 2023	Survey of hallucination.
Maynez 2020	Summaries often unfaithful; need a faithfulness check.
Liu 2024	Models lose the middle of long inputs.
Lewis 2020	RAG: retrieve then generate.
Hu 2022	LoRA: train a small add-on, not the whole model.
Tiedemann 2020	OPUS-MT: small open translation models.